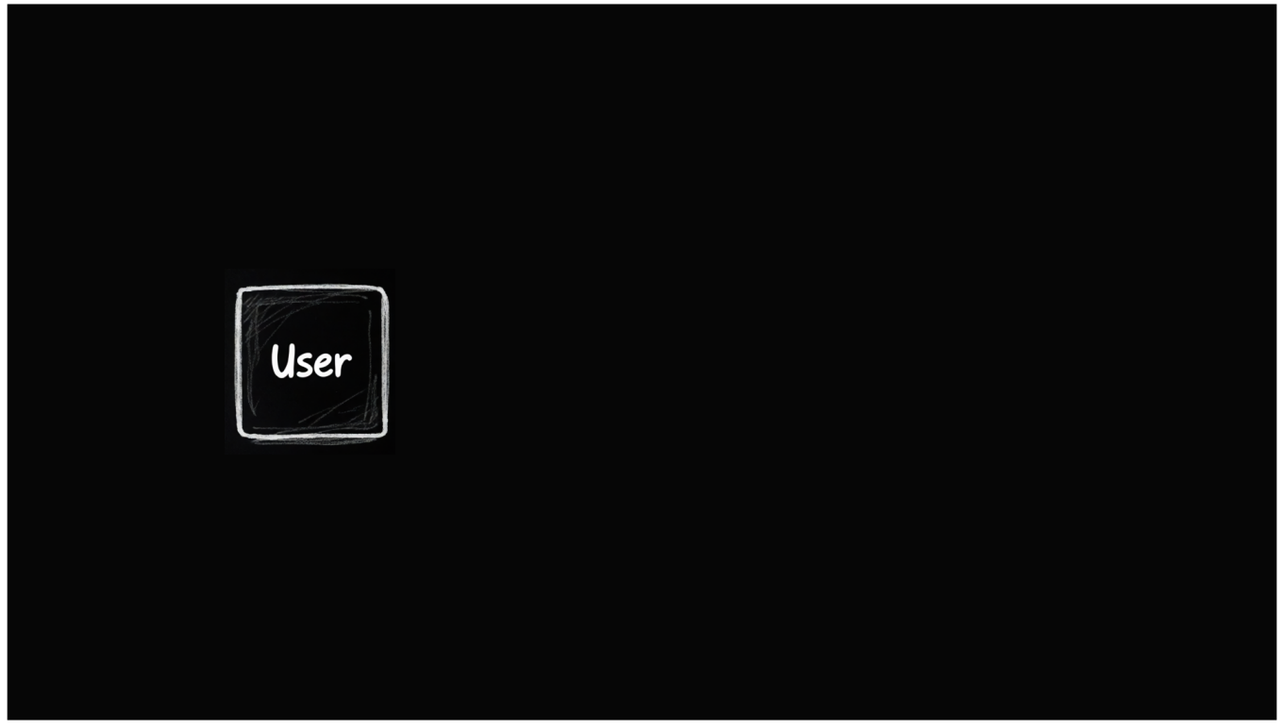
Behind The Prompt

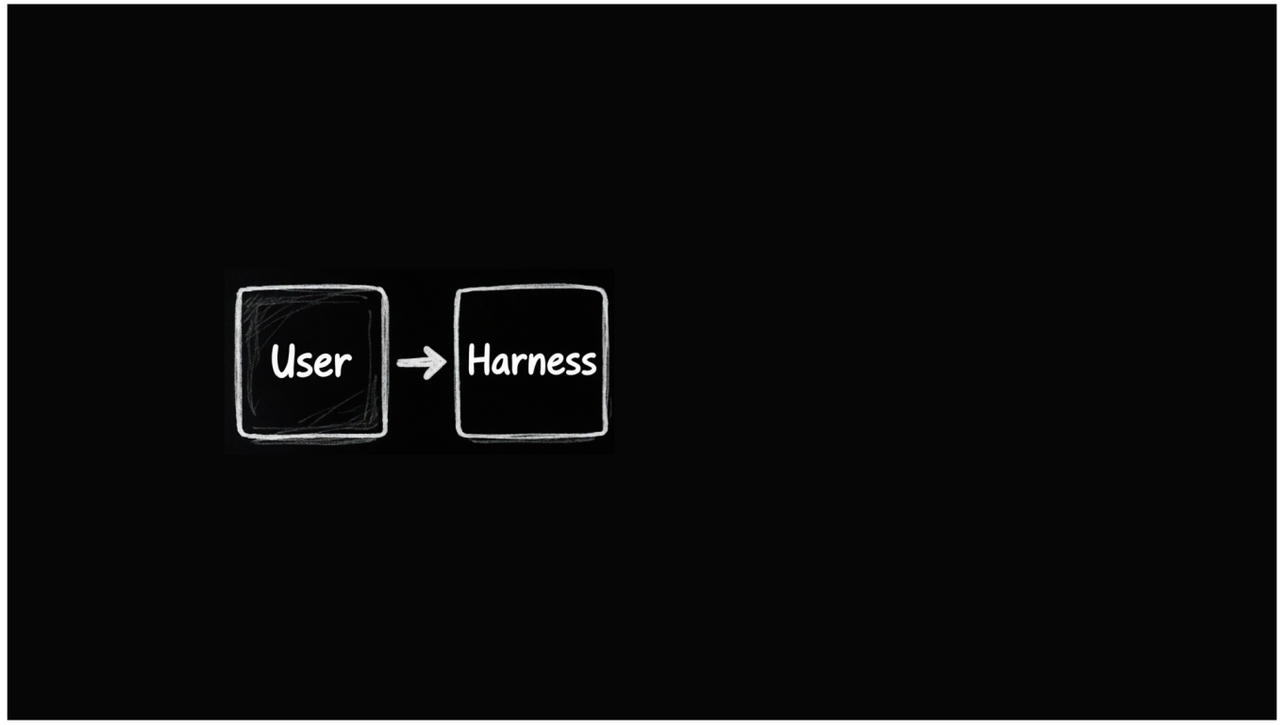
A Look Into AI Harnesses

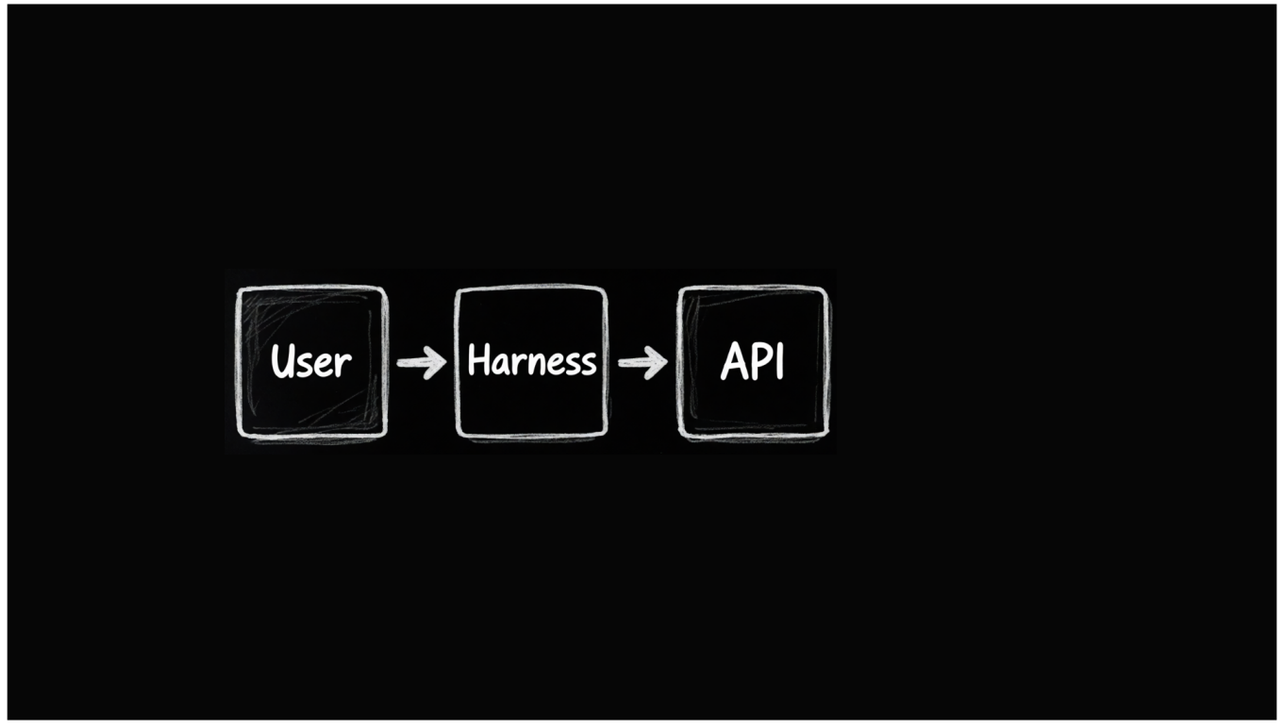
Christopher Primerano

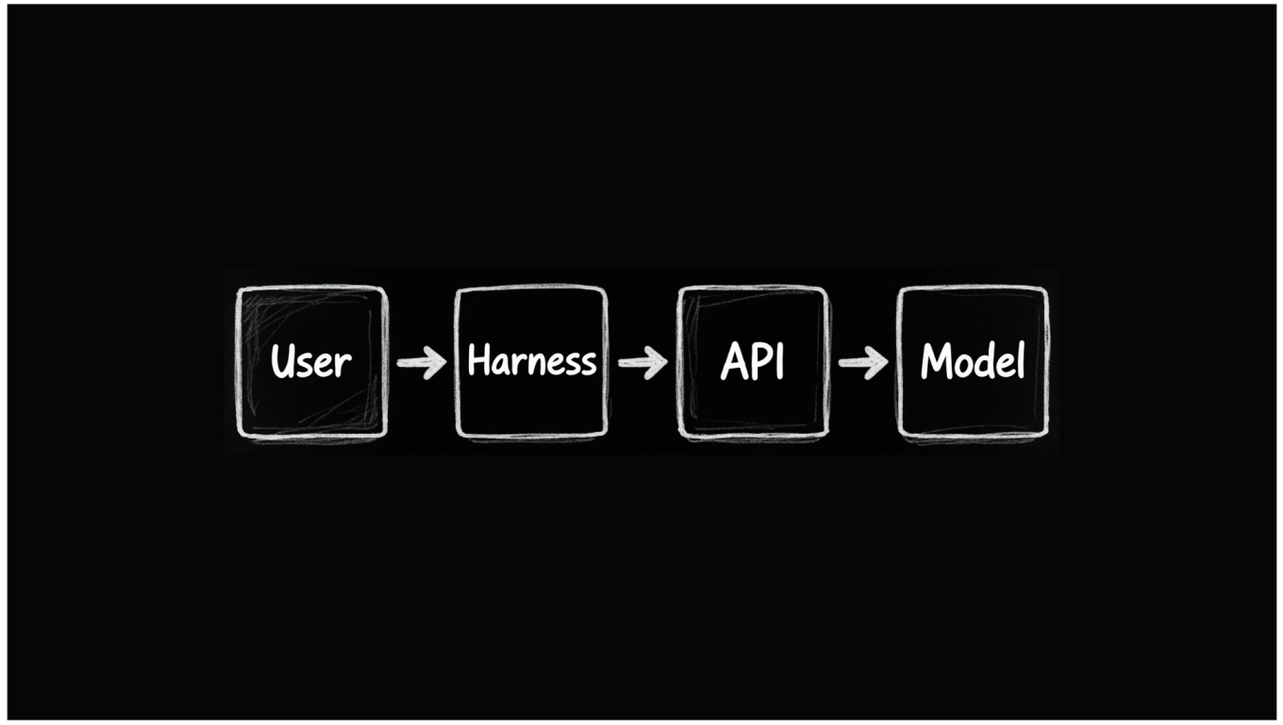


Rough AI System Architecture

* User interacts with a harness providing prompts
* Harness calls the provider API
* API loads the weights into memory, tokenizes the prompt, provides it to the model, model generates tokens response, response is de-tokenized and returned back to harness
* Harness inspects response, calls tools and continues loop if necessary and provides response to user


User
Harness

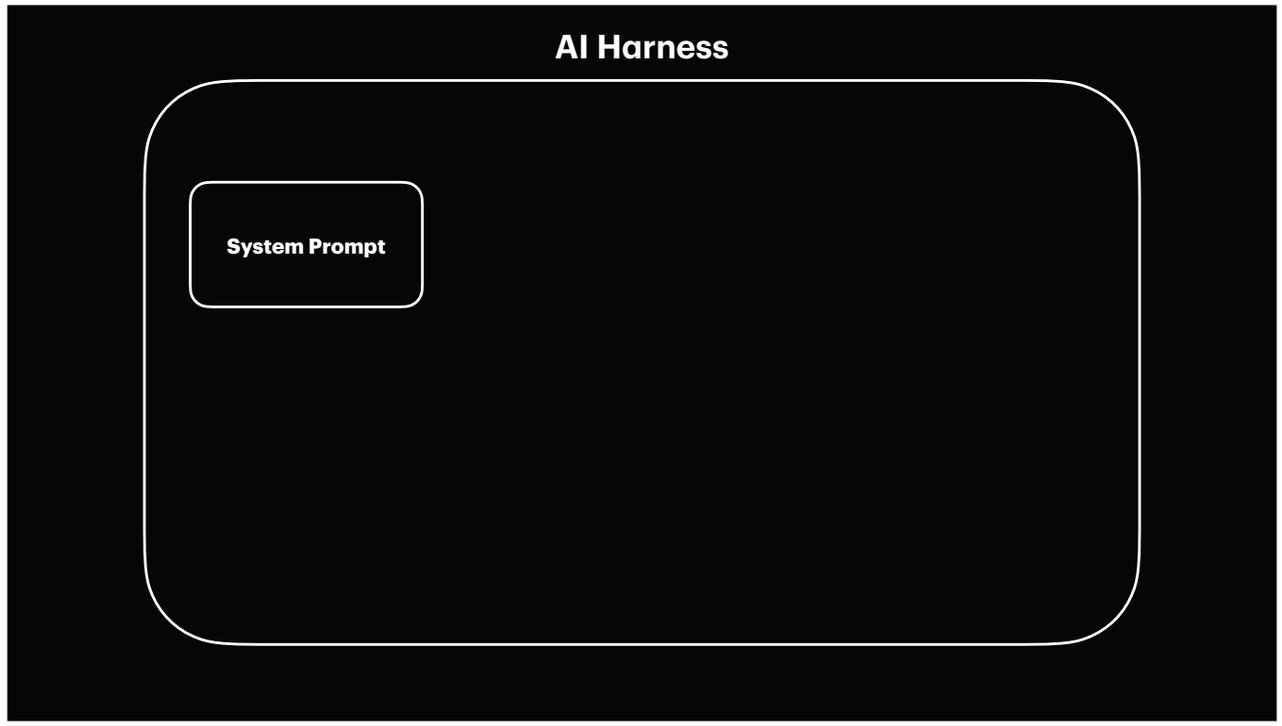

User
Harness
API

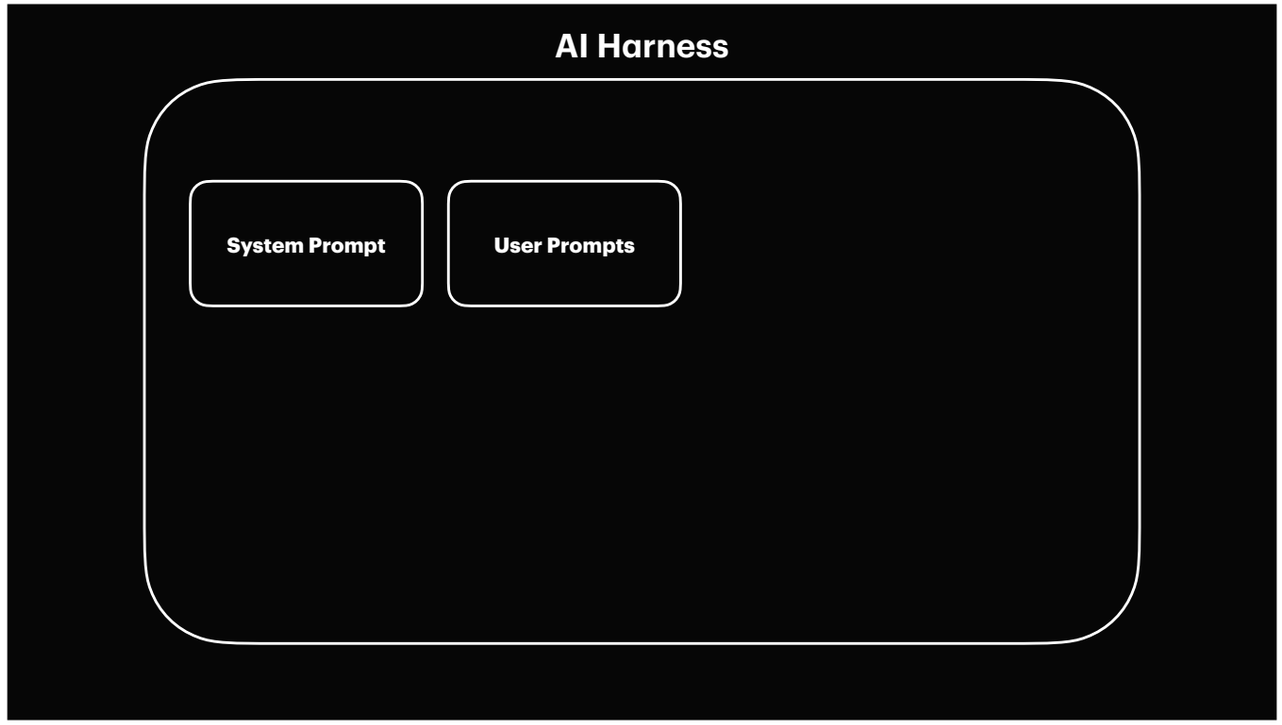

User
Harness
API
Model

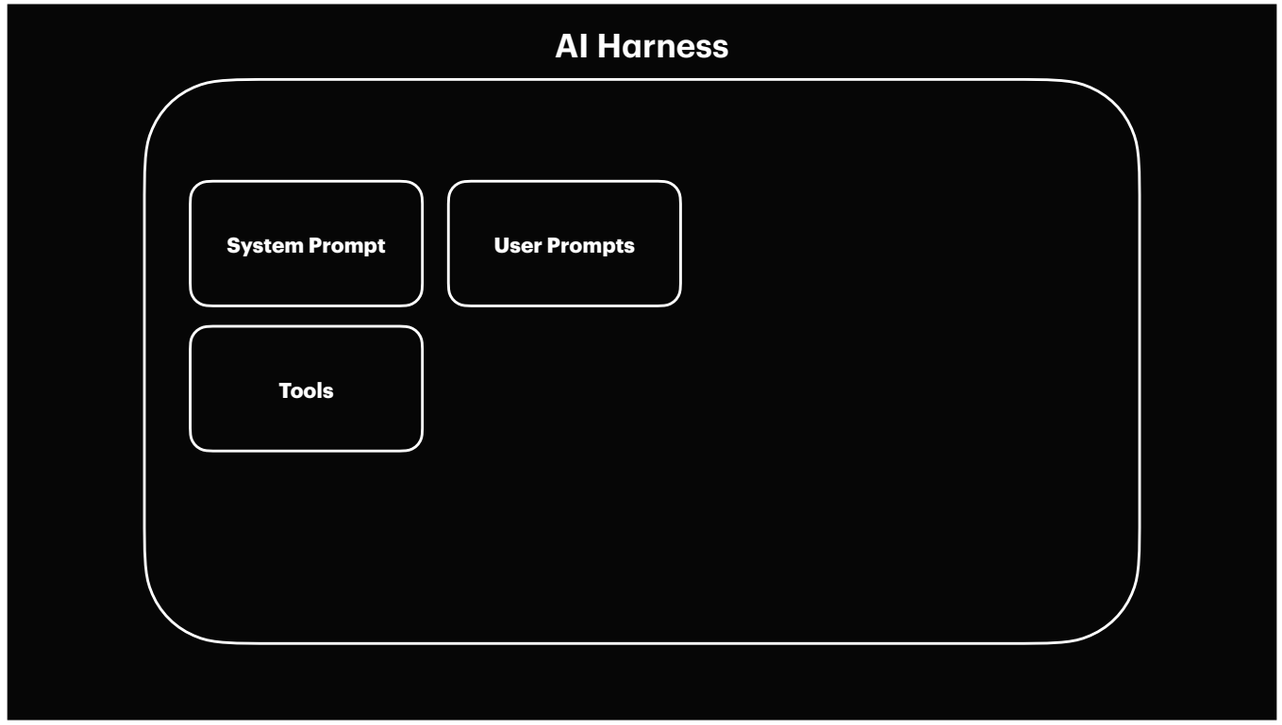


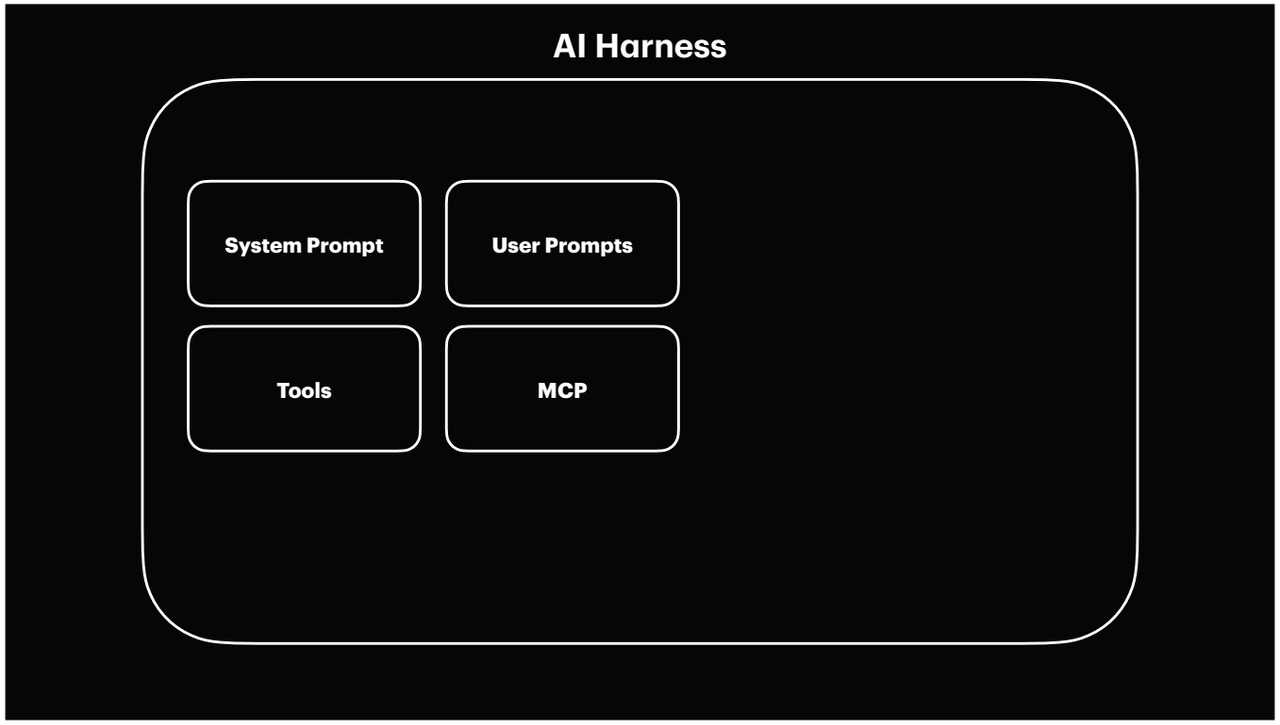
AI Harness Block Diagram

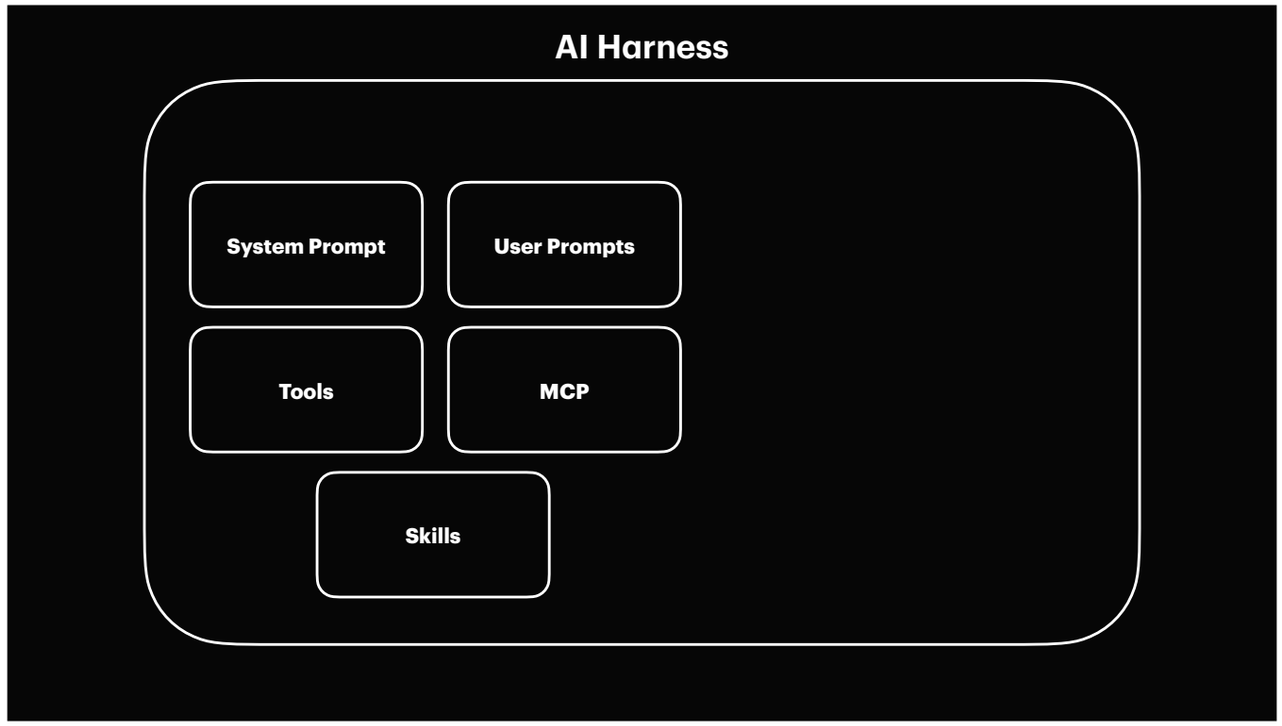
* System Prompt
* User Prompts
* Tools
* MCP
* Memory
* Skills

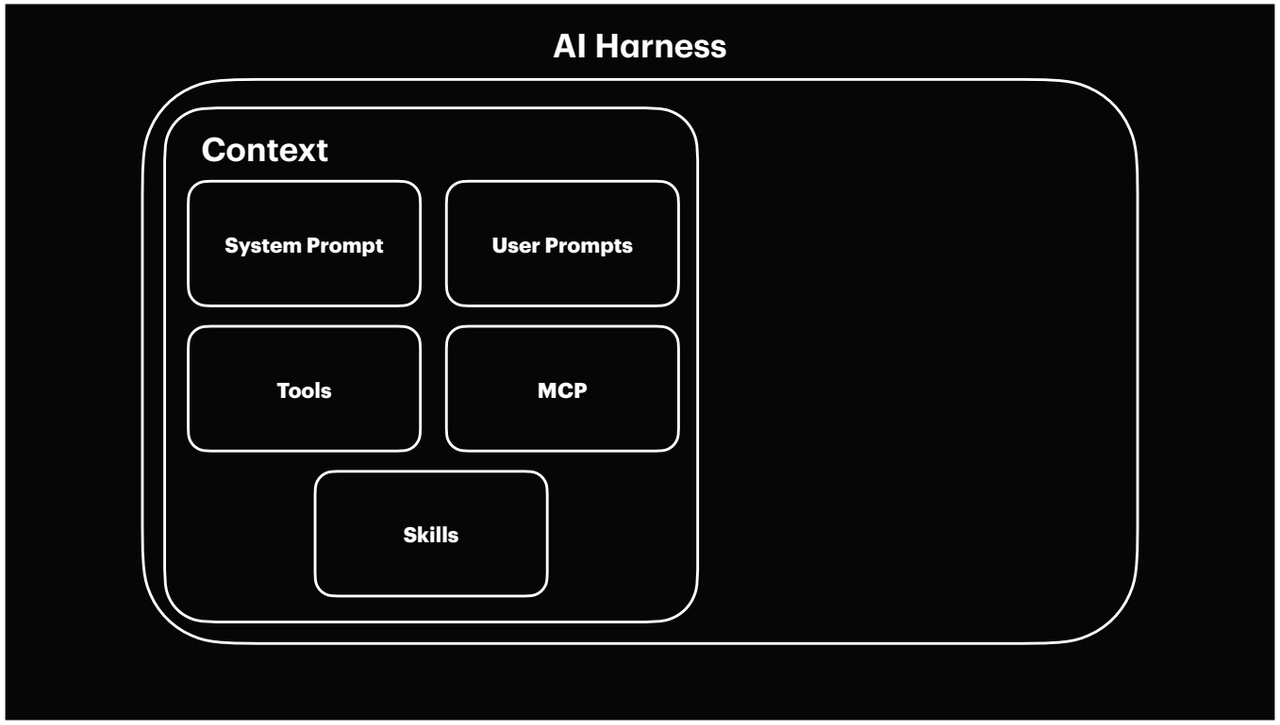

AI Harness
System Prompt

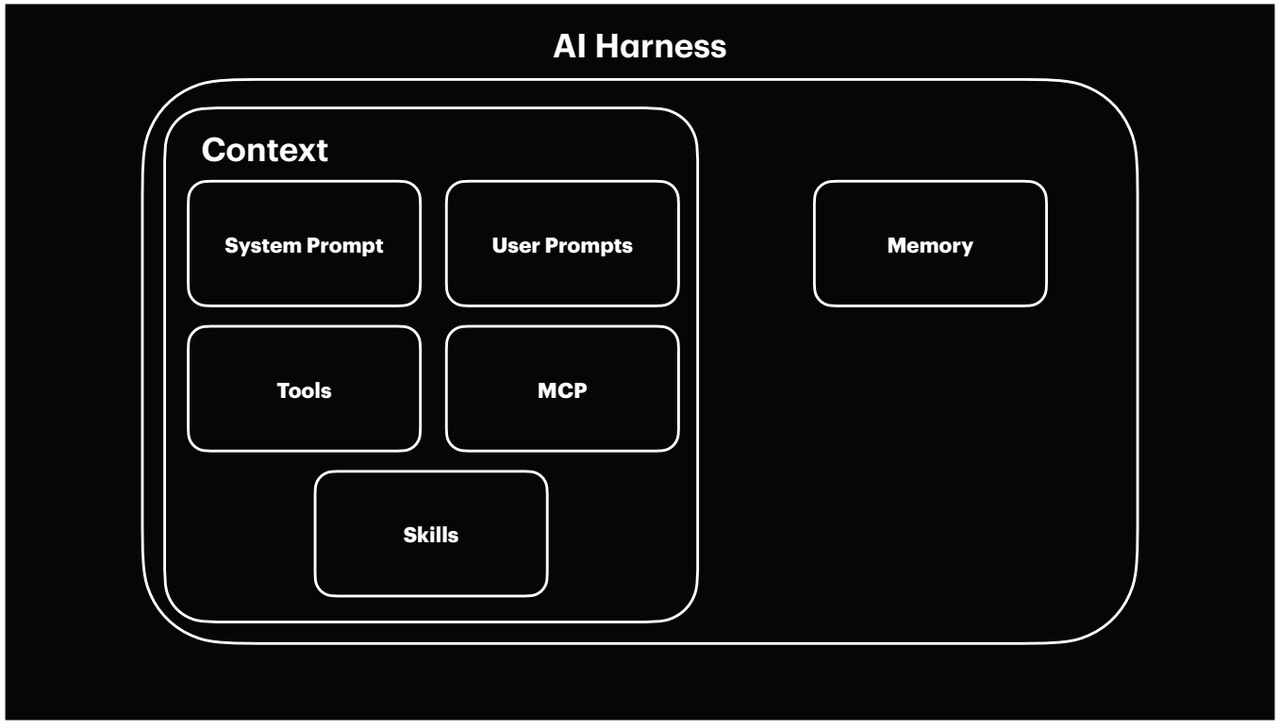

AI Harness
System Prompt
User Prompts

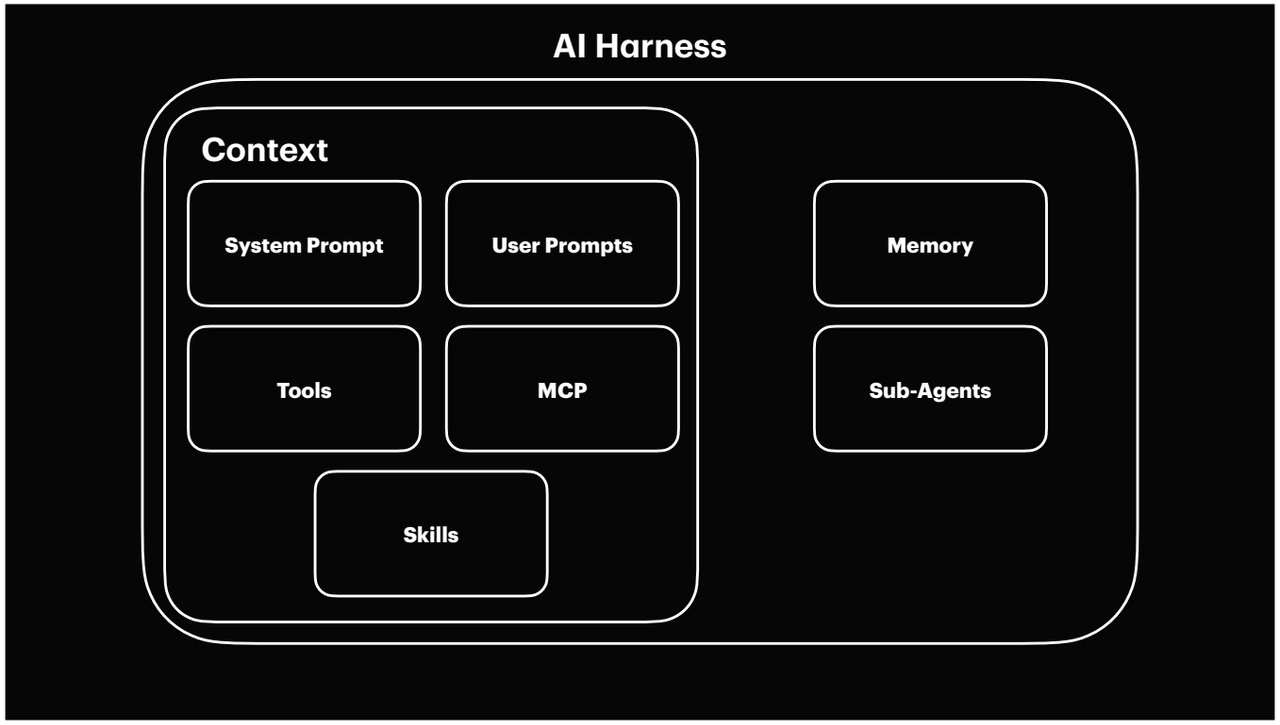

AI Harness
System Prompt
User Prompts
Tools

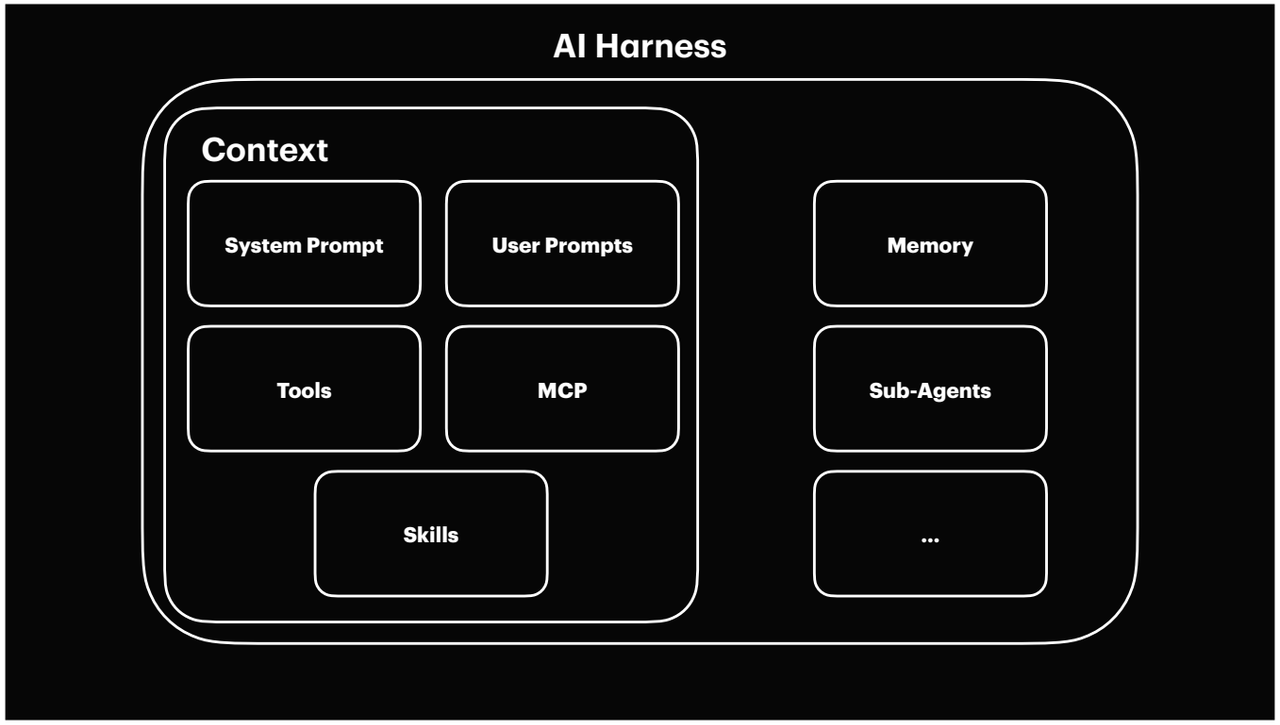

AI Harness
System Prompt
User Prompts
Tools
MCP


AI Harness
System Prompt
User Prompts
Tools
MCP
Skills


AI Harness
Context
System Prompt
User Prompts
Tools
MCP
Skills


AI Harness
Context
System Prompt
User Prompts
Memory
Tools
MCP
Skills


AI Harness
Context
System Prompt
User Prompts
Tools
MCP
Skills
Memory
Sub-Agents


AI Harness
Context
System Prompt
User Prompts
Tools
MCP
Skills
Memory
Sub-Agents
...



Use Copilot to make a call to LM Studio. Show logs with request/response. We're specifically going to look at the OpenAI chat completions API. There is also the newer responses API and Anthropic has their own messages API.

Most tooling now supports all of these to some degree. Chat completions is usually the best support in my experience

Request/Response Anatomy

Anatomy Of A Request

```
{
}
```

Deeper dive into request body

Anatomy Of A Request

```
{
    "model": "google/gemma-4-12b-qat"
}
```

Define the model we want to process the prompt

Anatomy Of A Request

```
{
    "model": "google/gemma-4-12b-qat",
    "max_tokens": 1024
}
```

We can specify the max number of tokens we want the model to generate

Anatomy Of A Request

```
{
    "model": "google/gemma-4-12b-qat",
    "max_tokens": 1024,
    "messages": [
    ]
}
```

All prompts from the context are included here

Anatomy Of A Request

```
{
    "model": "google/gemma-4-12b-qat",
    "max_tokens": 1024,
    "messages": [
        {"role": "system", "content": "You are a helpful assistant."}
    ]
}
```

First specify the "system prompt" by using the `system` role
With newer models we typically use the `developer` role now

Anatomy Of A Request

```
{
    "model": "google/gemma-4-12b-qat",
    "max_tokens": 1024,
    "messages": [
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Hello, how are you?"}
    ]
}
```

Next message is the user's prompt indicated by the `user` role



Since it's just an API we can make calls using cURL

```
curl --request POST \
  --url http://example.com:1234/v1/chat/completions \
  --header 'content-type: application/json' \
  --data '{"model": “model_name”,"max_tokens": 1024,"messages": [{"role": "system", "content": "You are a helpful assistant."},{"role": "user", "content": “Hello, how are you?”}]}'
```

Anatomy Of A Response

```
{
    "model": "google/gemma-4-12b-qat"
}
```

Deep dive into response format
Similar to the request we see the model that generated the response

Anatomy Of A Response

```
{
    "model": "google/gemma-4-12b-qat",
    "choices": [
    ]
}
```

The model can supply multiple choices of generated output. Rarely do you see multiple choices returned though

Anatomy Of A Response


```
{
    "model": "google/gemma-4-12b-qat",
    "choices": [
        {
            "message": {
            }
        }
    ]
}
```

Anatomy Of A Response

```
{
    "model": "google/gemma-4-12b-qat",
    "choices": [
        {
            "message": {
            }
        }
    ]
}
```

A choice has a message object which contains the generated content from the model

Anatomy Of A Response

```
{
    "model": "google/gemma-4-12b-qat",
    "choices": [
        {
            "message": {
                "role": "assistant"
            }
        }
    ]
}
```

It has a role, typically `assistant` meaning the model

Anatomy Of A Response


```
{
    "model": "google/gemma-4-12b-qat",
    "choices": [
        {
            "message": {
                "role": "assistant",
                "reasoning_content": "\nThe user said \"Hello, how are you?\"\nThis is a standard polite greeting.\n\n    *
Acknowledge the greeting (friendly and polite).\n    *   State status (as an AI, I don't have feelings/body but can express
functional well-being).\n    *   Reciprocate by asking the user how they are.\n    *   Offer assistance.\n\n    *   \"I'm doing
well, thank you! How can I help you today?\"\n    *   \"Hello! I'm doing great. How are you doing?\"\n    *   \"I'm well, thank you
for asking! How are things with you?\""
            }
        }
    ]
}
```


Since my model supports reasoning it also has `reasoning_content` which is the content it generated while "thinking"

Anatomy Of A Response


```
{
    "model": "google/gemma-4-12b-qat",
    "choices": [
        {
            "message": {
                "role": "assistant",
                "reasoning_content": "\nThe user said \"Hello, how are you?\"\nThis is a standard polite greeting.\n\n    *
Acknowledge the greeting (friendly and polite).\n    *   State status (as an AI, I don't have feelings/body but can express
functional well-being).\n    *   Reciprocate by asking the user how they are.\n    *   Offer assistance.\n\n    *   \"I'm doing
well, thank you! How can I help you today?\"\n    *   \"Hello! I'm doing great. How are you doing?\"\n    *   \"I'm well, thank you
for asking! How are things with you?\"",
                "content": "Hello! I'm doing well, thank you for asking. How are you doing today? Is there anything I can help you
with?"
            }
        }
    ]
}
```


It also has `content` which is the finalized generated output to the user

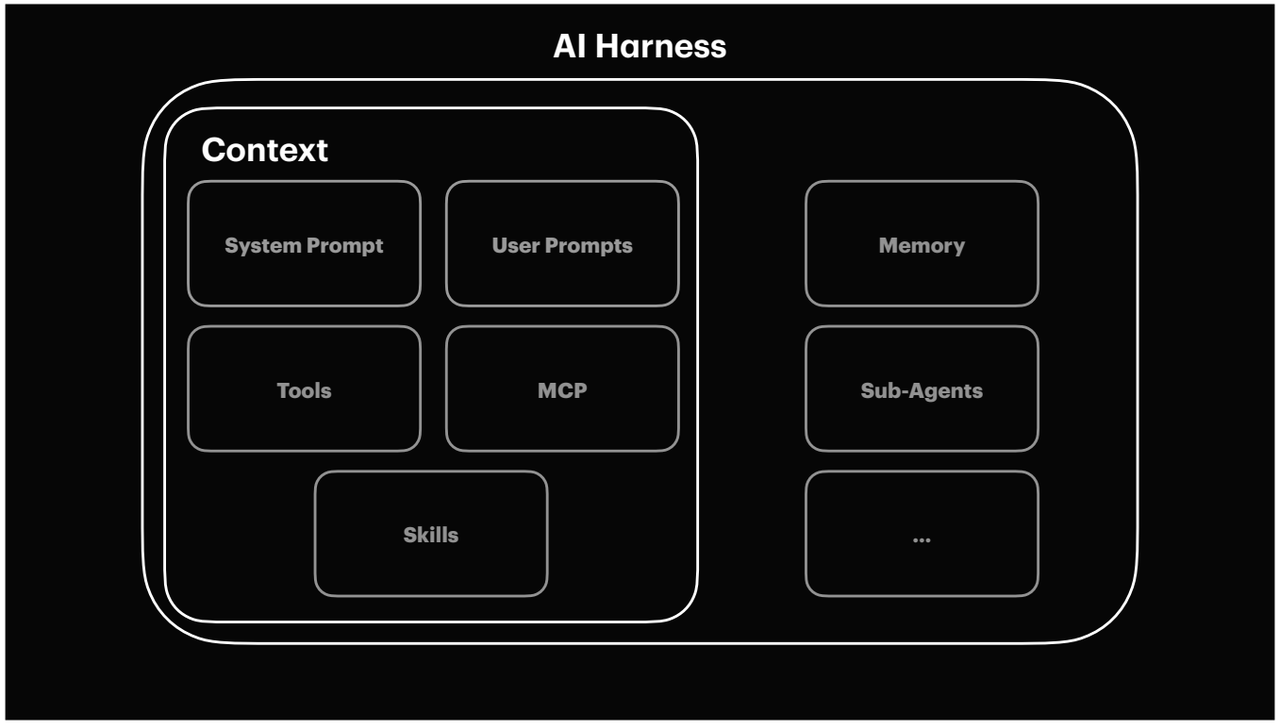
Anatomy Of A Response

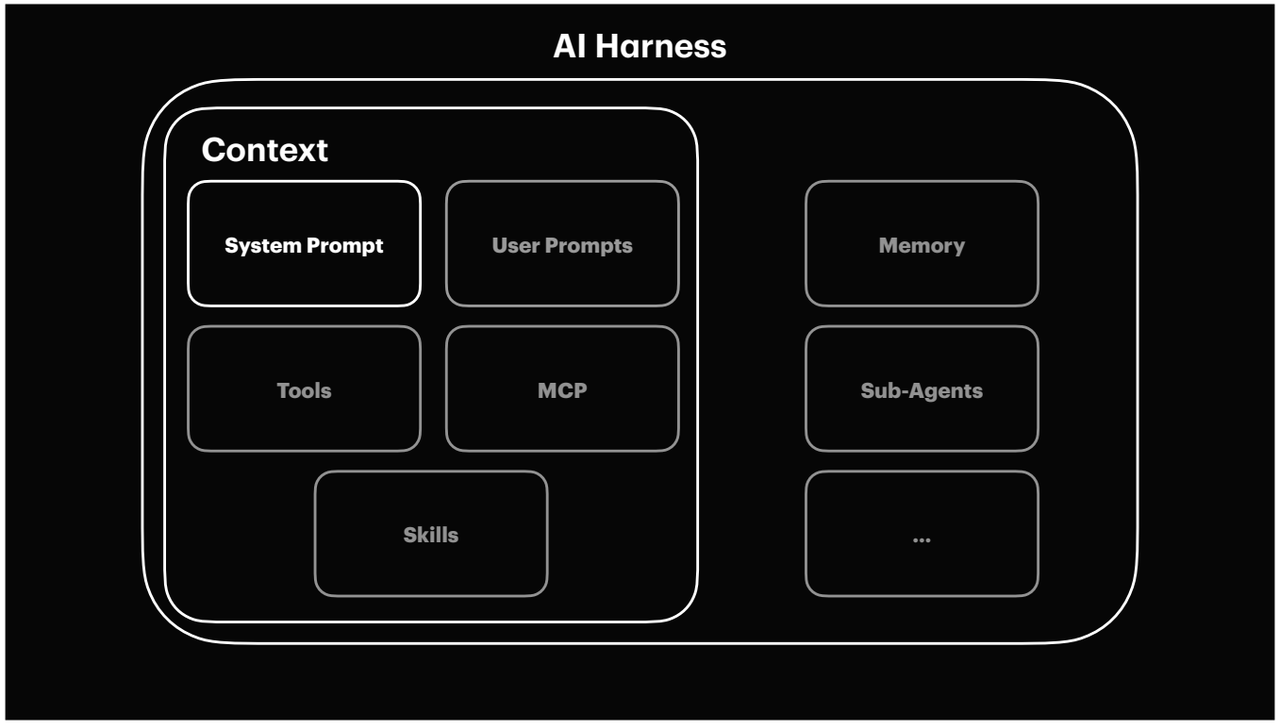

```
{
    "model": "google/gemma-4-12b-qat",
    "choices": [
        {
            "message": {
                "role": "assistant",
                "reasoning_content": "\nThe user said \"Hello, how are you?\"\nThis is a standard polite greeting.\n\n    *
Acknowledge the greeting (friendly and polite).\n    *   State status (as an AI, I don't have feelings/body but can express
functional well-being).\n    *   Reciprocate by asking the user how they are.\n    *   Offer assistance.\n\n    *   \"I'm doing
well, thank you! How can I help you today?\"\n    *   \"Hello! I'm doing great. How are you doing?\"\n    *   \"I'm well, thank you
for asking! How are things with you?\"",
                "content": "Hello! I'm doing well, thank you for asking. How are you doing today? Is there anything I can help you
with?"
            },
            "finish_reason": "stop"
        }
    ]
}
```

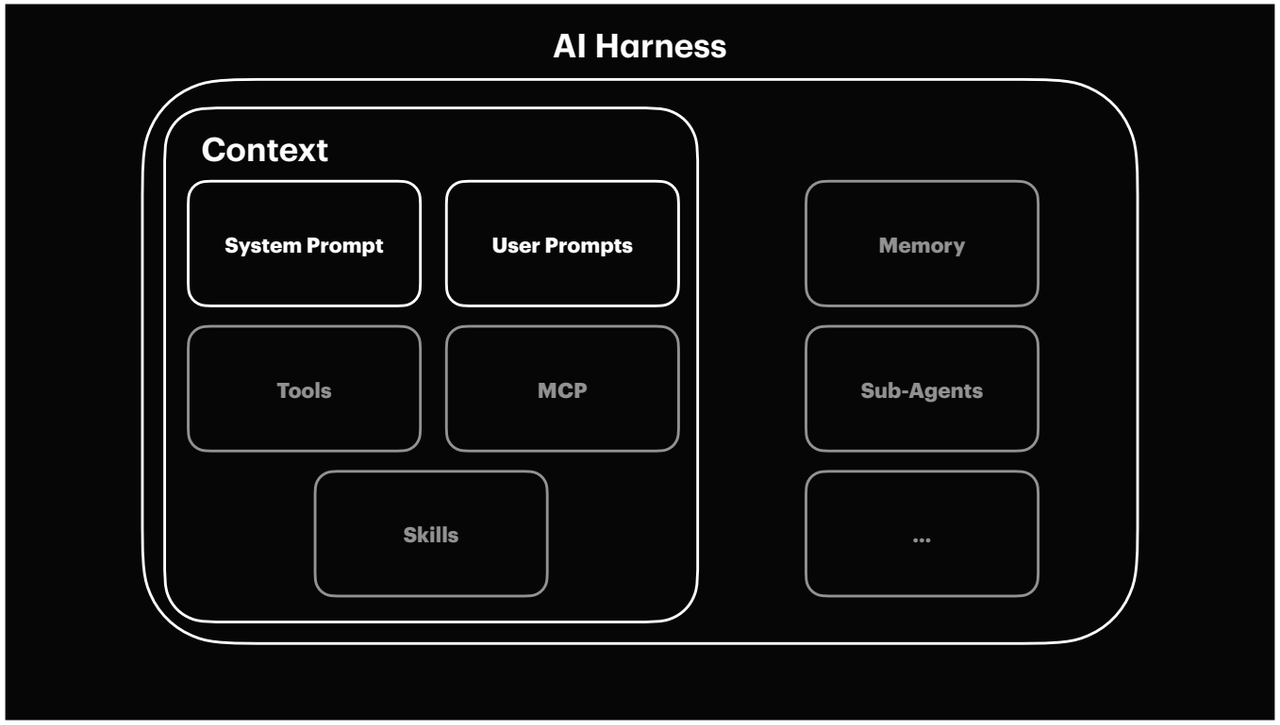

Finish reason can be:
`stop` - model is done generating tokens
`tool_calls` - Model wants to call a tool and is waiting for response
`length` - Hit max tokens



Start building the harness. ChatGPT level of interaction (prompt and response, no tools)


AI Harness
Context
System Prompt
User Prompts
Tools
MCP
Skills
Memory
Sub-Agents
...


AI Harness
Context
System Prompt
User Prompts
Tools
MCP
Skills
Memory
Sub-Agents
...


AI Harness
Context
System Prompt
User Prompts
Tools
MCP
Skills
Memory
Sub-Agents
...

Tool Definitions

Tool Definitions

```
{
    "model": "google/gemma-4-12b-qat",
    "max_tokens": 1024,
    "messages": [
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Hello, how are you?"}
    ]
}
```

Let's start with the request we were previously using

Tool Definitions

```
{
    "model": "google/gemma-4-12b-qat",
    "max_tokens": 1024,
    "messages": [
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Hello, how are you?"},
        {"role": "assistant", "content": "Hello! I'm doing well, thank you for asking. How are you doing today? Is there anything I can help you with?"}
    ]
}
```

We'll add in the response from the previous model call

Tool Definitions

```
{
    "model": "google/gemma-4-12b-qat",
    "max_tokens": 1024,
    "messages": [
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Hello, how are you?"},
        {"role": "assistant", "content": "Hello! I'm doing well, thank you for asking. How are you doing today? Is there anything I can help you with?"}
    ]
}
```

Tool Definitions

```
{
    "model": "google/gemma-4-12b-qat",
    "max_tokens": 1024,
    "messages": [
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Hello, how are you?"},
        {"role": "assistant", "content": "Hello! I'm doing well, thank you for asking. How are you doing today? Is there anything I can help you with?"}
    ],
    "tools": [
    ]
}
```

We now add a `tools` array to the request
Each tool definition has a type, and a JSON schema which defines the parameters it receives
In this case our `read_file` tool takes a single required `path` parameter which is the path to the file to read

Tool Definitions

```
{
    "model": "google/gemma-4-12b-qat",
    "max_tokens": 1024,
    "messages": [
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Hello, how are you?"},
        {"role": "assistant", "content": "Hello! I'm doing well, thank you for asking. How are you doing today? Is there anything I can help you with?"}
    ],
    "tools": [
        {
            "type": "function",
            "function": {
                "name": "read_file",
                "description": "Read a UTF-8 text file from disk and return its contents.",
                "parameters": {
                    "type": "object",
                    "properties": {
                        "path": {
                            "type": "string",
                            "description": "File path to read"
                        },
                    },
                    "required": ["path"]
                }
            }
        }
    ]
}
```

Tool Calls

Tool Calls


```
{
    "model": "google/gemma-4-12b-qat",
    "choices": [
        {
            "message": {
                "role": "assistant",
                "reasoning_content": "",
                "content": "",
            },
            "finish_reason": "stop"
        }
    ]
}
```


We now add a `tools` array to the request
Each tool definition has a type, and a JSON schema which defines the parameters it receives
In this case our `read_file` tool takes a single required `path` parameter which is the path to the file to read

Tool Calls


```
{
    "model": "google/gemma-4-12b-qat",
    "choices": [
        {
            "message": {
                "role": "assistant",
                "reasoning_content": "",
                "content": "",
                "tool_calls": [
                    {
                        "type": "function",
                        "id": "322392479",
                        "function": {
                            "name": "read_file",
                            "arguments": "{\"path\":\"./README.md\"}"
                        }
                    }
                ]
            },
            "finish_reason": "stop"
        }
    ]
}
```

Tool Calls


```
{
    "model": "google/gemma-4-12b-qat",
    "choices": [
        {
            "message": {
                "role": "assistant",
                "reasoning_content": "",
                "content": "",
                "tool_calls": [
                    {
                        "type": "function",
                        "id": "322392479",
                        "function": {
                            "name": "read_file",
                            "arguments": "{\"path\":\"./README.md\"}"
                        }
                    }
                ]
            },
            "finish_reason": "tool_calls"
        }
    ]
}
```


We now add a `tools` array to the request
Each tool definition has a type, and a JSON schema which defines the parameters it receives
In this case our `read_file` tool takes a single required `path` parameter which is the path to the file to read

Tool Call Response

Tool Call Response


```
{
    "model": "google/gemma-4-12b-qat",
    "max_tokens": 1024,
    "messages": [
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Hello, how are you?"},
        {"role": "assistant", "content": "Hello! I'm doing well, thank you for asking. How are you doing today? Is there anything I can help you with?"}
    ]
}
```

Tool Call Response

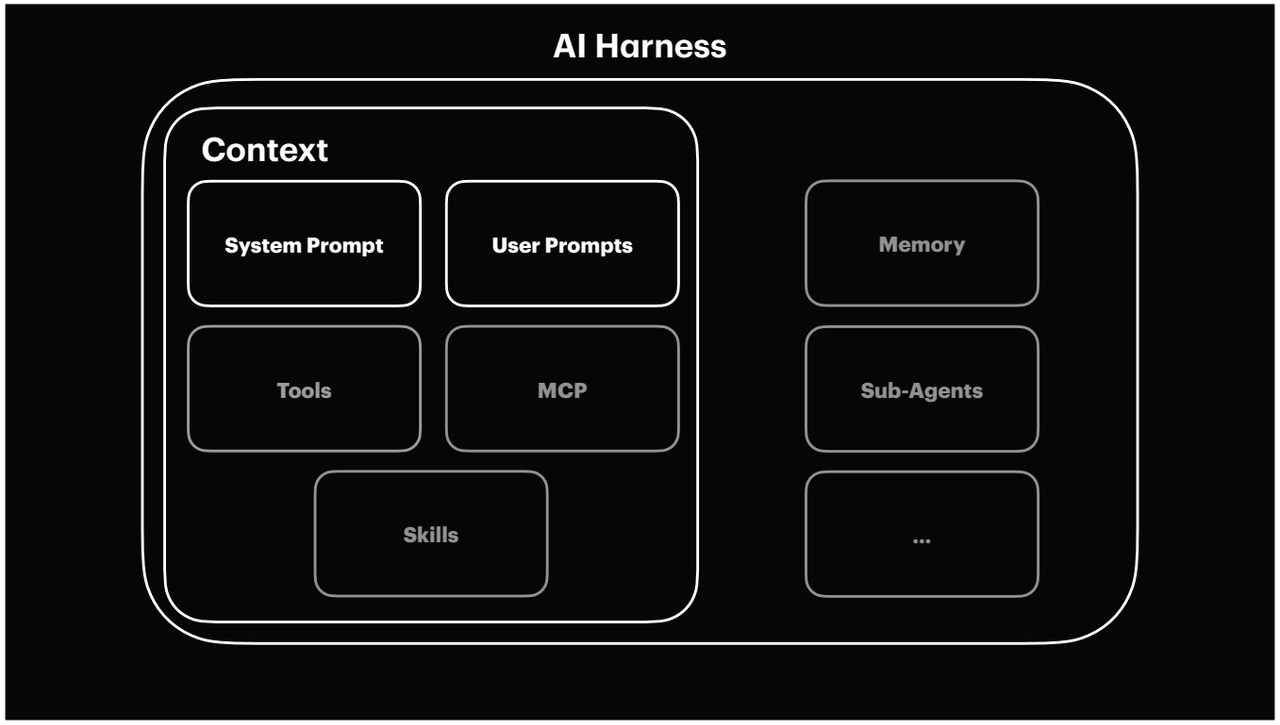

```
{
    "model": "google/gemma-4-12b-qat",
    "max_tokens": 1024,
    "messages": [
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Hello, how are you?"},
        {"role": "assistant", "content": "Hello! I'm doing well, thank you for asking. How are you doing today? Is there anything I can help you with?"},
        {"role": "assistant", "content": "", "reasoning_content": "", "tool_calls": [{"type": "function", "id": "322392479", "function": {"name":
"read_file", "arguments": "{\"path\":\"./README.md\"}"}}]}
    ]
}
```

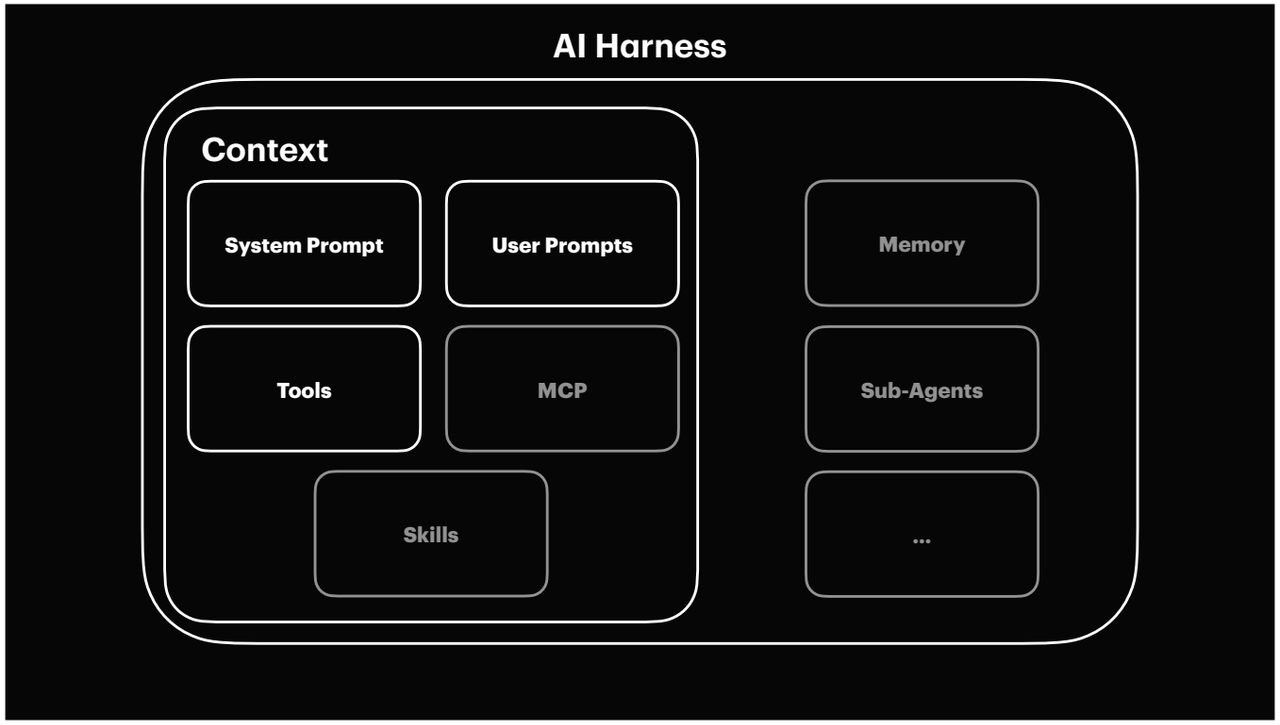
Tool Call Response


```
{
    "model": "google/gemma-4-12b-qat",
    "max_tokens": 1024,
    "messages": [
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Hello, how are you?"},
        {"role": "assistant", "content": "Hello! I'm doing well, thank you for asking. How are you doing today? Is there anything I can help you with?"},
        {"role": "assistant", "content": "", "reasoning_content": "", "tool_calls": [{"type": "function", "id": "322392479", "function": {"name":
"read_file", "arguments": "{\"path\":\"./README.md\"}"}}]},
        {"role": "tool", "tool_call_id": "322392479", "content": "{\"tool_name\": \"read_file\", \"path\": \"./README.md\", \"contents\": \"# Project
README\n\nAn overview of the project goes here.\", \"success\": true}"}
    ]
}
```



Add tools to harness


AI Harness
Context
System Prompt
User Prompts
Tools
MCP
Skills
Memory
Sub-Agents
...


AI Harness
Context
System Prompt
User Prompts
Tools
MCP
Skills
Memory
Sub-Agents
...

Takeaways

- Model is a stateless text generator
  - Every turn is a new request with the full context (system message, tools, history, etc.)
- Harness adds the interactions
- Sub-agents get their own context

Takeaways

* Model is stateless, harness adds all the interesting parts
* Full context sent on every request
* Use sub-agents, especially when the outcome is the only important/interesting part

Q&A